

Instructions conditionnelles dans Thymeleaf : th:if, th:unless, th:switch

1. th:if et th:unless

Principe

Dans certaines situations, on veut qu'un fragment du template n'apparaisse dans le résultat que si une condition est vraie. On utilise pour cela l'attribut **th:if**. Si la condition est fausse, l'élément **entier** (balise et contenu) est supprimé du HTML produit.

```
<someHtmlTag th:if="condition">
  <!-- Autre code -->
</someHtmlTag>

<!-- Ou, sans ajouter de balise HTML : -->
<th:block th:if="condition">
  <!-- Autre code -->
</th:block>
```

L'attribut **th:unless** est la négation de **th:if** :

```
<someTag th:unless="condition"> ... </someTag>

<!-- équivaut à -->
<someTag th:if="!condition"> ... </someTag>
```

Quand une condition est-elle vraie ou fausse ?

Dans Thymeleaf, une valeur est évaluée comme **fausse** si elle est null, le booléen false, le nombre 0, ou l'une des chaînes "false", "off" ou "no". Tous les autres cas sont évalués comme **vrais**.

Piège classique. Une chaîne vide "" et une collection vide sont évaluées comme **vraies**. Écrire `th:if="${product.image}"` affichera donc l'élément si image vaut "". Pour tester qu'une chaîne est renseignée, utilisez plutôt `th:if="${!#strings.isEmpty(product.image)}"`, et pour une collection `th:if="${!#lists.isEmpty(products)}"`.

Les conditions peuvent combiner des comparaisons (=, !=, <, >, ou lt, gt, le, ge, eq, ne) et des opérateurs logiques (and, or, ! ou not) :

```
<p th:if="${user.age} >= 18 and ${user.active}">Accès autorisé</p>
```

Exemple avec th:if

La classe du modèle (seuls le constructeur et les accesseurs sont nécessaires à Thymeleaf) :

```
package com.example.thymeleaf.model;

public class Product {

    private final Long id;
    private final String code;
    private final String name;
    private final String image; // nom du fichier image, peut être null

    public Product(Long id, String code, String name, String image) {
        this.id = id;
        this.code = code;
        this.name = name;
        this.image = image;
    }

    public Long getId() { return id; }
    public String getCode() { return code; }
    public String getName() { return name; }
    public String getImage() { return image; }
}
```

Le contrôleur :

```
@GetMapping("/if-example1")
public String ifExample1(Model model) {
    List<Product> products = List.of(
        new Product(1L, "SS-S9", "Samsung Galaxy S9", "samsung-s9.png"),
        new Product(2L, "NK-5P", "Nokia 5.1 Plus", null),
        new Product(3L, "IP-7", "iPhone 7", "iphone-7.jpg"));

    model.addAttribute("products", products);
    return "if-example1";
}
```

Le template if-example1.html :

```
<!DOCTYPE html>
<html xmlns:th="http://www.thymeleaf.org">
<head>
    <meta charset="UTF-8" />
    <title>If / Unless</title>
    <style>
        .product-container {
            padding: 5px; border: 1px solid #ccc;
            margin-top: 10px; height: 80px;
        }
    </style>
</head>
<body>
    <div th:if="${products.size() > 0}" class="product-container">
        <table border="1">
            <thead>
                <tr>
                    <th>ID</th>
                    <th>Code</th>
                    <th>Name</th>
                    <th>Image</th>
                </tr>
            </thead>
            <tbody>
                <tr>
                    <td>1</td>
                    <td>SS-S9</td>
                    <td>Samsung Galaxy S9</td>
                    <td><img alt="Samsung Galaxy S9" data-bbox="680 780 800 800" /></td>
                </tr>
                <tr>
                    <td>2</td>
                    <td>NK-5P</td>
                    <td>Nokia 5.1 Plus</td>
                    <td><img alt="Nokia 5.1 Plus" data-bbox="680 810 800 830" /></td>
                </tr>
                <tr>
                    <td>3</td>
                    <td>IP-7</td>
                    <td>iPhone 7</td>
                    <td><img alt="iPhone 7" data-bbox="680 840 800 860" /></td>
                </tr>
            </tbody>
        </table>
    </div>
</body>
</html>
```

```

    .img-container { float: left; margin-right: 5px; }
  </style>
</head>
<body>
  <h1>th:if</h1>

  <div class="product-container" th:each="product : ${products}">

    <!--/* Commentaire Thymeleaf : supprimé du HTML produit.
       Si le produit a une image, ce bloc est affiché. */-->
    <div class="img-container"
      th:if="${!#strings.isEmpty(product.image)}">
      
    </div>

    <div><b>Code :</b> <span th:text="${product.code}"></span></div>
    <div><b>Nom :</b> <span th:text="${product.name}"></span></div>
  </div>
</body>
</html>

```

Remarques :

- Les images sont placées dans `src/main/resources/static/images/`.
- L'URL est construite avec une **variable de chemin** : `@{/images/{file}(file=${product.image})}`. Thymeleaf se charge d'encoder correctement la valeur, ce qui est plus sûr que la concaténation `@{|/${product.image}|}`.
- La syntaxe `<!--/* ... */-->` est un **commentaire au niveau de l'analyseur** (*parser-level comment*) : il est supprimé du résultat, contrairement à un commentaire HTML ordinaire `<!-- ... -->` qui est envoyé au navigateur.
- Sur une même balise, `th:each` est évalué avant `th:if` : un `th:if` placé à côté d'un `th:each` peut donc utiliser la variable de boucle.

Exemple avec `th:if` et `th:unless`

On ajoute une image par défaut lorsque le produit n'en a pas. Le contrôleur est identique ; seul le template change :

```

<div class="product-container" th:each="product : ${products}">

  <!--/* Le produit a une image : on l'affiche. */-->
  <div class="img-container" th:if="${!#strings.isEmpty(product.image)}">
    
  </div>

  <!--/* Sinon : image par défaut. */-->

```

```
<div class="img-container" th:unless="${!#strings.isEmpty(product.image)}">
  
</div>

<div><b>Code :</b> <span th:text="${product.code}"></span></div>
<div><b>Nom :</b> <span th:text="${product.name}"></span></div>
</div>
```

On utilise ici deux fois la même condition. Pour ne pas la dupliquer, on peut la définir une fois avec `th:with`, sur un `<th:block>` placé à l'intérieur de la boucle :

```
<th:block th:with="hasImage = ${!#strings.isEmpty(product.image)}">
  <div class="img-container" th:if="${hasImage}"> ... </div>
  <div class="img-container" th:unless="${hasImage}"> ... </div>
</th:block>
```

th:if, opérateur conditionnel ou Elvis ?

Besoin	Outil recommandé
Afficher ou masquer un bloc entier	<code>th:if</code> / <code>th:unless</code>
Choisir entre deux valeurs (texte, classe CSS, ...)	Opérateur conditionnel <code>cond ? a : b</code>
Fournir une valeur par défaut si null	Opérateur Elvis <code>valeur ?: défaut</code>

```
<!-- Valeur par défaut si l'âge est null -->
<p>Âge : <span th:text="${person.age} ?: '(âge non renseigné)'">27</span>.</p>
```

Pour plus de détails, voir le document sur les opérateurs conditionnel et Elvis.

2. th:switch et th:case

Principe

En Java, vous connaissez la structure `switch / case`. Thymeleaf propose une structure équivalente : `th:switch` et `th:case`.

```
<div th:switch="${user.role}">
  <p th:case="'admin'">L'utilisateur est administrateur</p>
  <p th:case="${roles.manager}">L'utilisateur est manager</p>
  <p th:case="'staff'">L'utilisateur fait partie du personnel</p>
</div>

<!-- Avec un cas par défaut : -->
<div th:switch="${user.role}">
  <p th:case="'admin'">L'utilisateur est administrateur</p>
  <p th:case="${roles.manager}">L'utilisateur est manager</p>
  <p th:case="'staff'">L'utilisateur fait partie du personnel</p>
  <p th:case="*">Autre type d'utilisateur</p>
</div>
```

Fonctionnement :

- Les `th:case` sont évalués **de haut en bas**. Dès qu'un cas correspond à la valeur de `th:switch`, son contenu est affiché et **tous les autres cas sont ignorés** (il n'y a pas de passage au cas suivant, contrairement au `switch` Java sans `break`).
- Un `th:case` peut comparer la valeur à un littéral (`'admin'`) ou à une variable (`${roles.manager}`).
- **`th:case=""`** est le cas par défaut : il est affiché si aucun des cas précédents ne correspond. Il doit donc être placé en dernier.

Exemple complet

Le modèle :

```
package com.example.thymeleaf.model;

public class User {

    private final String userName;
    private final String role;

    public User(String userName, String role) {
        this.userName = userName;
        this.role = role;
    }

    public String getUserName() { return userName; }
    public String getRole() { return role; }
}
```

Le contrôleur :

```
@GetMapping("/switch-example")
public String switchExample(Model model) {
    model.addAttribute("user", new User("Administrator", "admin"));
    return "switch-example";
}
```

Le template `switch-example.html` :

```
<!DOCTYPE html>
<html xmlns:th="http://www.thymeleaf.org">
<head>
    <meta charset="UTF-8" />
    <title>th:switch / th:case</title>
</head>
<body>
    <h1>th:switch / th:case</h1>

    <h4 th:text="${user.userName}"></h4>
```

```
<div th:switch="${user.role}">
  <p th:case="'admin'">L'utilisateur est administrateur</p>
  <p th:case="'manager'">L'utilisateur est manager</p>
  <p th:case="'staff'">L'utilisateur fait partie du personnel</p>
  <p th:case="*">Autre type d'utilisateur</p>
</div>
</body>
</html>
```

Avec le rôle admin, seul le premier paragraphe apparaît dans le HTML produit.

À retenir. Contrairement à `th:if`, la comparaison de `th:switch` porte sur l'égalité entre la valeur du switch et celle du case, et non sur la véracité d'une expression. On ne peut donc pas écrire de condition comme `th:case="${age} > 18"` : dans ce cas, utilisez une suite de `th:if`.

Aller plus loin : sécurité et affichage selon les rôles

Afficher un bloc selon le rôle de l'utilisateur ne doit **jamais** être la seule protection d'une fonctionnalité : un bouton masqué dans la page n'empêche pas d'appeler directement l'URL correspondante. La vraie vérification doit être faite côté serveur (Spring Security, `@PreAuthorize`). Avec le module `thymeleaf-extras-springsecurity6`, on dispose de l'attribut `sec:authorize` pour masquer l'interface en cohérence avec ces règles :

```
<a sec:authorize="hasRole('ADMIN')" th:href="@{/admin}">Administration</a>
```

Résumé

- `th:if` affiche l'élément si la condition est vraie ; `th:unless` fait l'inverse.
- Sont faux : `null`, `false`, `0`, `"false"`, `"off"`, `"no"`. Une chaîne vide et une collection vide sont **vraies** : utilisez `#strings.isEmpty` et `#lists.isEmpty`.
- `th:each` est évalué avant `th:if` sur une même balise.
- `th:switch` / `th:case` compare par égalité ; un seul cas est affiché ; `th:case="*" est le cas par défaut, à placer en dernier.`
- `<!--/* ... */-->` est un commentaire supprimé du HTML produit.
- Masquer un élément dans la vue ne remplace pas un contrôle d'accès côté serveur.

Exercices proposés

1. Afficher un message « Aucun produit » lorsque la liste `products` est vide, et le tableau des produits sinon.
2. Modifier l'exemple pour que le produit dont l'image est une chaîne vide `""` reçoive aussi l'image par défaut. Que se passe-t-il avec `th:if="${product.image}"` ?
3. Afficher en rouge le prix d'un produit en rupture de stock grâce à `th:if`, puis refaire l'exercice avec `th:classappend` et l'opérateur conditionnel.

4. Écrire un `th:switch` sur le statut d'une commande (NEW, PAID, SHIPPED) avec un cas par défaut.
5. Expliquer pourquoi masquer un lien « Supprimer » dans la page ne suffit pas à protéger l'action de suppression, et indiquer où placer le contrôle.